Registro de decisiones (anexo A de la propuesta)

Versión de entrega del 2026-11-04. Las fichas F-01 a F-15 consolidan decisiones tomadas en las carpetas de cada integrante; F-16 a F-25 se cerraron al integrar la propuesta (`entrega/propuesta_boletaya.pdf`) y la hoja de costos (`entrega/costos_boletaya_final.xlsx`). **Mínimo exigido:** 8 fichas con contexto, alternativas, criterio, elección y sacrificio. Aquí hay **25 fichas**; las decisiones están tomadas y las verificaciones de compra, costos o terceros que siguen pendientes se indican en cada ficha. **Regla de esta carpeta:** no se copió ni se cambió nada de las carpetas de los demás. Cada ficha dice de dónde sale.

#	Decisión	Dueño	Estado
F-01	Escenario B híbrido (DynamoDB no va *on premise*)	Jacobo	Cerrada (profesor, resp. 1)
F-02	Región principal us-east-1, alterna us-east-2	Jacobo	Cerrada, sujeta a Ley 1581 (Samuel)
F-03	Ocho tablas especializadas en vez de tabla única	Jacobo	Cerrada (D-1)
F-04	Clave de sillas por evento, localidad y fila, sin GSI	Jacobo	Cerrada
F-05	Contador exacto fragmentado dentro de la transacción	Jacobo, David	Cerrada (D-5)
F-06	Control optimista: escritura condicional + transacción; el TTL no libera	David	Cerrada (D-9)
F-07	Capacidad on-demand con pre-calentamiento	Jacobo, Juan Pablo	Cerrada (D-4)
F-08	Mapa de sillas por CDN; la base solo para comprar	Jacobo	Cerrada
F-09	Una sola región escribe; RPO 0 de pagos por conciliación	David, Miguel	Cerrada (D-2, R-1; profesor resp. 3)
F-10	`bya_ledger` como fuente de las sillas en duda	David, Jacobo	Cerrada (D-2b)
F-11	Pasarela de pagos: Mercado Pago	David, Miguel, Jacobo	Cerrada (D-11)
F-12	Respaldo inmutable en otra región y otra cuenta	Miguel	Cerrada en diseño; costo abierto (R-2)
F-13	Estadio: un nodo por puerta como autoridad	David	Cerrada
F-14	Equipo de puerta: usar los torniquetes del estadio	David, Juan Pablo	Cerrada
F-15	Cola virtual propia sobre AWS	Jacobo, Juan Pablo	Decidida; estimación y prueba de carga pendientes de validar
F-16	Reserva del 10 %: solo B la usa	Juan Pablo	Cerrada
F-17	AWS Backup semanal y mensual (PITR cubre los 35 días)	Miguel, Juan Pablo	Cerrada
F-18	Escenario B: VPN sobre dos enlaces dedicados en vez de Direct Connect	Jacobo, Juan Pablo	Cerrada
F-19	Comisión de la pasarela fuera del presupuesto de infraestructura	Juan Pablo	Cerrada
F-20	PCI: tokenización reduce alcance; SAQ sujeto al adquirente	Samuel	Diseño definido; SAQ por confirmar
F-21	Retención por clase: firmeza tributaria o 10 años mercantiles	Samuel, Miguel	Diseño definido; matriz por validar
F-22	Recomendación final: Escenario A	Grupo	Cerrada
F-23	12 terminales portátiles en vez de 24	David, Juan Pablo	Cerrada

#	Decisión	Dueño	Estado
F-24	Fondo de reposición del hardware del estadio	Juan Pablo	Cerrada
F-25	B sin caché en espera en us-east-2	Jacobo, Miguel	Cerrada

---

F-01. Escenario B híbrido

Campo	Contenido
Contexto	El enunciado pide un Escenario B con la base en el data center de Bogotá. DynamoDB es un servicio administrado de AWS: DynamoDB Local es solo para desarrollo y pruebas, y Outposts no lo ejecuta localmente
Alternativas	(a) DynamoDB Local en producción. (b) Un motor *on premise* compatible con la API de DynamoDB. (c) Híbrido: aplicación, caché y estadio en Bogotá; base en AWS por Direct Connect y VPN
Criterio	Usar el motor asignado sin inventar una configuración que no existe; que la verdad transaccional quede en el motor asignado
Elección	(c). El profesor la aceptó el 2026-11-04 y pidió una sección de máximo 2 páginas que explique por qué no se puede *on premise* y evalúe (b) sin diseñarla
Sacrificio	B depende del enlace Bogotá–AWS: si cae, no se vende. Añade Direct Connect, VPN redundante y una aplicación en espera en AWS. Cada compra suma unos 353 ms de ida y vuelta frente a unos 22 ms en A
Fuente	`jacobo/arquitectura.md` §3; `jacobo/resumen_sustentacion.md` P10 y P18; `david/respuestas_profesor.md` resp. 1
Pendiente	Sección de 2 páginas (Jacobo)

F-02. Región us-east-1 con alterna us-east-2

Campo	Contenido
Contexto	No se ha confirmado que AWS tenga una región con DynamoDB en Colombia (verificación pedida por el profesor). Hay que elegir dónde vive la base
Alternativas	(a) sa-east-1 (São Paulo). (b) us-east-1 (Virginia) con us-east-2 (Ohio) como alterna. (c) Otra región norteamericana
Criterio	Latencia desde Bogotá, precio, disponibilidad de funciones (MRSC) y viabilidad legal
Elección	(b). Ping medido por un tercero desde Bogotá: 77,1 ms a Washington frente a 112,2 ms a São Paulo. DynamoDB cuesta un 50 % más en São Paulo, que además no tiene MRSC
Sacrificio	Los datos personales quedan fuera de Colombia y exigen el análisis de transferencia internacional de la Ley 1581. La latencia es un ping de un tercero, no una medición contra AWS
Fuente	`jacobo/arquitectura.md` §1; `jacobo/calculos/evidencia/latencia_wondernetwork_bogota_2026-10-29.txt`
Pendiente	Ley 1581 (Samuel); infraestructura de AWS en Colombia y latencia (Jacobo); plan B sa-east-1 si se rechaza

F-03. Ocho tablas en vez de una

Campo	Contenido
Contexto	La guía de DynamoDB recomienda “tan pocas tablas como sea posible”, salvo cuando los patrones de acceso son muy distintos
Alternativas	(a) Tabla única. (b) Ocho tablas especializadas (`bya_seats`, `bya_orders`, `bya_counters`, `bya_tickets`, `bya_catalog`, `bya_users`, `bya_seat_history`, `bya_audit`)
Criterio	Retención, carga, permisos y respaldo distintos por tipo de dato; las transacciones sí funcionan entre tablas
Elección	(b), con tabla única solo donde hay un agregado (orden + pagos + factura en `bya_orders`)
Sacrificio	Más objetos que operar y configurar (PITR, Streams, alarmas por tabla); sin `JOIN`
Fuente	`jacobo/modelo_datos.md` §2; aprobada por David (D-1)

F-04. Clave de sillas por evento, localidad y fila

Campo	Contenido
Contexto	En el pico casi toda la escritura cae en las sillas de un evento. Una partición acepta como máximo 1.000 escrituras por segundo
Alternativas	(a) Clave por evento. (b) Clave por localidad. (c) Clave por evento, localidad y fila. (d) Cualquiera de ellas más un GSI por localidad
Criterio	Que ninguna partición se acerque a 1.000 WRU/s en el pico de planificación ×5 (≈ 2.214 WRU/s sobre las sillas)
Elección	(c), sin GSI: unas 900 claves por estadio y unos 2,5 WRU/s por clave en promedio
Sacrificio	Leer una localidad completa exige una consulta por fila; la disponibilidad por localidad sale de instantáneas, no de un índice
Fuente	`jacobo/modelo_datos.md` §3.2 y §4.1; `jacobo/resumen_sustentacion.md` P1

F-05. Contador exacto fragmentado dentro de la transacción

Campo	Contenido
Contexto	Los organizadores exigen un contador exacto por localidad, actualizado con cada venta, y en preventa miles de compras tocan las mismas pocas localidades
Alternativas	(a) Un contador único dentro de la transacción. (b) Contador derivado con Streams. (c) Contador fragmentado (N = 8) dentro de la transacción
Criterio	Exacto con cada venta (no solo al converger) y con el menor riesgo de choque entre transacciones
Elección	(c): se vende y se cuenta en la misma transacción; el total es la suma de 8 fragmentos con lectura fuerte
Sacrificio	Leer el total cuesta 8 lecturas. La frescura no es un instante único salvo con `TransactGetItems`. La frecuencia real de conflictos no está medida
Fuente	`jacobo/modelo_datos.md` §5; `jacobo/resumen_sustentacion.md` P2

F-06. Control optimista con escritura condicional y transacción

Campo	Contenido
Contexto	Regla de oro: una silla nunca se vende dos veces. DynamoDB no tiene niveles de aislamiento SQL ni bloqueos que esperen
Alternativas	(a) Bloqueo pesimista en una tabla aparte. (b) Escritura condicional sobre el ítem de la silla dentro de `TransactWriteItems` (todo o nada), con el bloqueo de 8 minutos como atributo (`lx`, `lo`). (c) Liberar sillas vencidas con TTL
Criterio	Cero doble venta demostrable y sin depender de procesos que tardan
Elección	(b). La silla vencida se trata como disponible dentro de la condición; el TTL solo limpia. Parámetros: máximo 10 sillas por orden, `:skew` = 2 s, *backoff* con *jitter* y máximo 3 reintentos
Sacrificio	Las transacciones cuestan el doble de capacidad, también al cancelarse; un choque cancela la transacción completa y el cliente reintenta
Fuente	`david/concurrencia.md` §2–§5; `david/anexo_concurrencia.md`; prueba en simulador: 50 de 50 casos pasan (`david/pruebas/resultado.txt`)

F-07. Capacidad on-demand con pre-calentamiento

Campo	Contenido
Contexto	El presupuesto no alcanza para pagar todo el año la capacidad de la preventa
Alternativas (throughput del año 1, USD)	(a) On-demand: 83 (us-east-1) a 124 (sa-east-1). (b) Provisionado con escalado programado: 880 a 1.319. (c) Provisionado fijo al pico: 63.176 a 94.764. (d) Auto scaling reactivo
Criterio	Costo anual y capacidad de absorber un salto súbito sin rechazar compras
Elección	(a) con pre-calentamiento (*warm throughput*) antes de cada preventa; (b) como plan B. (d) se descarta: dispara tras 2 minutos y tarda varios más
Sacrificio	Si el tráfico supera el doble del pico previo en 30 minutos sin pre-calentar, puede haber *throttling*. El monto exacto del pre-calentamiento no está verificado (Juan Pablo supone USD 13 como cargo único)
Fuente	`jacobo/capacidad_y_volumetria.md` §7; `jacobo/resumen_sustentacion.md` P3 y P12; `juanpablo/costos_y_dimensionamiento.md` §4.2

F-08. Mapa de sillas por CDN

Campo	Contenido
Contexto	250.000 personas en la cola consultan el mapa. Sondeando cada 10 s serían 25.000 consultas por segundo, 17,2 veces la cuota de una tabla
Alternativas	(a) Leer el mapa de la base. (b) Caché con DAX o ElastiCache. (c) Instantáneas en CloudFront regeneradas cada 2 s desde la base
Criterio	Que la consulta masiva no toque la base principal y que ninguna venta dependa de la copia
Elección	(c). La base ve unos 450 RRU/s del constructor de instantáneas. La compra siempre se valida con la transacción
Sacrificio	El mapa puede ir unos segundos atrasado (el profesor lo acepta si la validación final es contra la base). CloudFront cuesta USD 25, 76 o 253 por preventa (moderado, alto, extremo)
Fuente	`jacobo/capacidad_y_volumetria.md` §6; `david/concurrencia.md` §0

F-09. Una sola región escribe; RPO 0 de pagos por conciliación

Campo	Contenido
Contexto	El enunciado exige RPO 0 para pagos. MRSC (RPO 0 entre regiones) no admite transacciones ni TTL; MREC es asíncrono y, si dos regiones escriben, “la última escritura gana” puede vender una silla dos veces
Alternativas	(a) MREC con dos regiones escribiendo. (b) MRSC para toda la compra. (c) MREC con una sola región que escribe y conciliación contra la pasarela. (d) Aceptar un RPO mayor que 0
Criterio	RPO 0 del dinero sin romper la atomicidad de la compra ni la regla de oro
Elección	(c). Ante la pérdida de región se congelan solo las sillas en duda, se reabre y la conciliación termina después. El profesor lo aceptó con tres condiciones, todas atendidas: atraso de replicación con fuente (típicamente ≤ 1 s, diseño para 60 s con alarma), sillas en duda congeladas y RTO dentro de 15 minutos
Sacrificio	La base queda con RPO de segundos ante pérdida de región; el RPO 0 de pagos depende de un proceso, no de la réplica. RTO estimado de 4 a 11 minutos desde la declaración (más 2 a 5 de detección), **sin medir**: solo el simulacro lo confirma. Si pasara de 15, se automatiza el cambio de tráfico con Application Recovery Controller (costo por cotizar)
Fuente	`david/alta_disponibilidad.md` §2–§5; `miguel/fichas_decision_miguel.md` R-1; `david/respuestas_profesor.md` resp. 3 y 6
Pendiente	Conformidad de Samuel

F-10. `bya_ledger` como fuente de las sillas en duda (D-2b)

Campo	Contenido
Contexto	El plan era sacar la lista de sillas en duda de la búsqueda por fechas de Mercado Pago. En el sandbox esa búsqueda devolvió 0 resultados, incluso 14 minutos después
Alternativas	(a) Confiar en la búsqueda por fechas en producción sin haberla visto funcionar. (b) Libro de pagos `bya_ledger` en MRSC, escrito antes de cobrar. (c) Reembolsar todo pago de desastre sin sillas conocidas
Criterio	No descubrir en pleno desastre que la fuente de la lista no sirve
Elección	(b), aprobada por el grupo el 2026-11-04. Cada pago se consulta luego en Mercado Pago por su `id`
Sacrificio	Una escritura sincrónica entre regiones en el camino del pago (latencia sin medir); tabla MRSC en tres regiones con unas 24.000 escrituras por preventa grande (costo por calcular)
Fuente	`david/pasarelas.md` §7; `david/alta_disponibilidad.md` §5.0
Pendiente	Clave listable por minuto y medición de latencia (Jacobo); costo (Juan Pablo)

F-11. Pasarela de pagos: Mercado Pago

Campo	Contenido
Contexto	La conciliación necesita idempotencia, metadatos en el cobro y una forma de consultar pagos
Alternativas	Mercado Pago, Wompi, PayU Latam, ePayco (documentación no accesible), Stripe (no disponible para empresas en Colombia)
Criterio	Idempotencia que devuelva el mismo pago, metadatos propios en el cobro, consulta de pagos y webhook firmado
Elección	Mercado Pago. En el sandbox (11 de 13 pruebas) se confirmó que `metadata` guarda y devuelve las sillas y que la misma `X-Idempotency-Key` devuelve el mismo pago
Sacrificio	La búsqueda por fechas no funcionó en el sandbox (de ahí F-10). Comisión de referencia 3,29 % + COP 800 por cobro + IVA, sin confirmar en la cuenta
Fuente	`david/pasarelas.md` §1, §2 y §7; `david/pruebas/resultados_sandbox.json`

F-12. Respaldo inmutable en otra región y otra cuenta

Campo	Contenido
Contexto	PITR protege de errores, pero no de quien tenga acceso a la cuenta y borre tablas y respaldos. El enunciado pide una copia inmutable o desconectada
Alternativas	(a) Solo PITR. (b) PITR + AWS Backup en la misma cuenta y región. (c) PITR + AWS Backup con copia a otra región y otra cuenta, en una bóveda con Vault Lock
Criterio	Sobrevivir a una cuenta comprometida con un costo que quepa en el presupuesto
Elección	(c), con Vault Lock probado antes en modo gobernanza
Sacrificio	El modo cumplimiento es irreversible y compromete el costo hasta que vence la retención; la copia diaria deja hasta 24 h en riesgo ante una cuenta comprometida; hay que operar dos cuentas
Fuente	`miguel/fichas_decision_miguel.md` R-2; `miguel/recuperacion_licenciamiento_carta.md` §5.5
Pendiente	Costo real de AWS Backup con incrementalidad (Juan Pablo; la hoja `Respaldo` solo tiene la cota superior)

F-13. Estadio: un nodo por puerta como autoridad

Campo	Contenido
Contexto	El ingreso no puede detenerse sin internet (hasta 4 horas aislado, según el profesor) y una boleta no puede entrar dos veces
Alternativas	(a) Cada lector decide solo con una copia local. (b) Un servidor central único. (c) Cada boleta tiene una puerta dueña y cada puerta un nodo que marca la boleta con un `UPDATE` atómico
Criterio	Cero doble ingreso sin depender de internet ni de la conexión entre puertas
Elección	(c), con dos servidores centrales en réplica síncrona y sincronización idempotente con DynamoDB al volver la conexión
Sacrificio	Asignación fija de puerta (no se puede desviar gente a otra puerta durante el evento). Cambiar un nodo caído exige que una persona confirme que está apagado (5 a 10 min, supuesto). Se cierran los reembolsos 2 h antes de abrir puertas
Fuente	`david/estadio.md` §1, §6 y §8

F-14. Equipo de puerta: torniquetes del estadio

Campo	Contenido
Contexto	Con 48 torniquetes propios (COP 216,1 M, solo el cuerpo), el Escenario B suma COP 336,6 M en el año 1 frente a un tope de 280 M, y todavía faltan cotizaciones
Alternativas	(1) 48 torniquetes propios. (2) Lectores QR que abren los torniquetes que ya tiene el estadio. (3) Alquilar torniquetes por evento. (4) Carriles con lector portátil y vallas. (5) Comprar 24 torniquetes
Criterio	Caber en el presupuesto sin cambiar la regla contra el doble ingreso (la autoridad es el nodo, no el torniquete)
Elección	(2) como base, (4) como respaldo en estadios sin torniquetes compatibles, (1) solo si el cliente exige equipo propio. Con (1), B superaba el tope del año 1 (COP 336,6 M)
Sacrificio	(2) depende de integrar equipos ajenos y de cada estadio; (4) no tiene barrera física y exige más personal
Fuente	`david/estadio.md` §9.2
Pendiente	La pregunta 45 al profesor no tiene respuesta; si el cliente exige equipo propio, A lo absorbe con su margen y B no

F-15. Cola virtual propia sobre AWS

Campo	Contenido
Contexto	250.000 personas (610.352 en el año 5) esperan en una sala de espera. El producto de AWS para esto está descontinuado. El estado de la cola vive en AWS para no perder el turno si cae Bogotá
Alternativas	(a) Construcción propia sobre servicios de AWS. (b) Producto comercial
Criterio	Costo, SLA y que ninguna venta dependa solo de la cola
Elección	(a): tokens de admisión firmados emitidos por Lambda, posición como objeto cacheable en CloudFront y estado en ElastiCache Multi-AZ (patrón de SeatGeek [R-30]). Su costo ya está en las líneas de API, Lambda, ElastiCache y CloudFront
Sacrificio	Hay que construirla y probarla (no hay producto de AWS con SLA); un producto comercial no tiene precio público
Fuente	`jacobo/entrega_para_el_equipo.md` D-8; `juanpablo/costos_y_dimensionamiento.md` §6
Pendiente	Prueba de carga antes de la primera preventa

F-16. Reserva del 10 %: solo B la usa

Campo	Contenido
Contexto	El enunciado permite hasta un 10 % adicional sobre el año 1, una sola vez, con una solicitud de adición presupuestal
Alternativas	(a) No usarla. (b) Usarla para cubrir variaciones verificadas de precios
Criterio	Solo si el total cotizado supera el presupuesto base y no lo supera con la reserva
Elección	A no la usa (sobran COP 88,9 M en el año 1). B se pasa COP 9,6 M en el año 1 y la usa con la solicitud de adición presupuestal de `juanpablo/sensibilidad_y_tco.md` §7; le quedan 18,4 M de margen
Fuente	Borrador de solicitud en `juanpablo/sensibilidad_y_tco.md` §7

F-17. AWS Backup semanal y mensual

Campo	Contenido
Contexto	Miguel propuso copias diarias de AWS Backup con 35 días de retención, además de PITR de 35 días. Sin incrementalidad confirmada, la cota superior crece con cada copia completa
Alternativas	(a) Diaria 35 días + mensual 13 meses. (b) Semanal (5) + mensual (13), solo tablas críticas, en bóveda con Vault Lock y copia a otra cuenta y región. (c) Solo PITR
Criterio	Cubrir una cuenta comprometida sin duplicar lo que PITR ya da
Elección	(b). PITR da cualquier segundo de los últimos 35 días; la copia inmutable es para el caso en que la cuenta se pierda
Sacrificio	Ante una cuenta comprometida, la copia inmutable puede tener hasta 7 días de atraso (antes 24 h). Cuesta ≈ COP 3,2 M en el año 1 y 15,2 M en el año 5
Fuente	`entrega/costos_boletaya_final.xlsx`, línea AWS Backup; `miguel/fichas_decision_miguel.md` R-2

F-18. Escenario B: VPN sobre dos enlaces dedicados

Campo	Contenido
Contexto	La arquitectura de B usaba Direct Connect en Equinix BG1, pero el puerto, el *cross-connect* y el circuito no tienen precio publicado: no se pueden costear con fuente
Alternativas	(a) Direct Connect + VPN de respaldo. (b) Dos enlaces dedicados a internet de proveedores distintos (1 Gbps y 512 Mbps) con una VPN Site-to-Site sobre cada uno
Criterio	Redundancia real y precio con fuente
Elección	(b): COP 26,9 M al año con IVA (precios de la orden de compra 160603 de Colombia Compra Eficiente) + USD 876 al año de VPN
Sacrificio	La latencia por internet varía más que por Direct Connect; Direct Connect queda como mejora si la latencia medida fuera inaceptable
Fuente	`entrega/propuesta_boletaya.pdf` §2.3

F-19. Comisión de la pasarela fuera del presupuesto de infraestructura

Campo	Contenido
Contexto	La hoja anterior incluía la comisión de Mercado Pago (3,29 % + COP 800 por cobro), sin valor promedio de orden
Alternativas	(a) Incluirla en el presupuesto de la base de datos. (b) Tratarla como costo de recaudo que se descuenta del ingreso
Criterio	El enunciado enumera lo que debe incluir el costo (§4) y no menciona comisiones de pago
Elección	(b); se reporta aparte cuando haya valor promedio de orden
Sacrificio	El costo total del negocio por boleta no queda en esta hoja

F-20. PCI DSS: tokenización y alcance reducido

Campo	Contenido
Contexto	Guardar números de tarjeta en cualquier tabla, log o respaldo cuesta −1,0
Alternativas	(a) Formulario propio que envía la tarjeta a la pasarela desde el servidor. (b) Formulario seguro de la pasarela en el navegador, que devuelve un token
Criterio	Que la tarjeta nunca toque servidores, base, logs ni respaldos de BoletaYa
Elección	(b). La tokenización reduce el alcance: solo se guardan `id` y token; se excluyen cuerpos de logs y `metadata` contiene identificadores. No se declara automáticamente fuera de PCI; el adquirente confirma el SAQ aplicable antes de producción
Sacrificio	Menos control sobre la experiencia de pago
Fuente	`entrega/propuesta_boletaya.pdf` §8.3; validar con adquirente/asesor PCI

F-21. Retención de órdenes, pagos y boletas

Campo	Contenido
Contexto	El enunciado pide 2 años en línea y luego el plazo de la norma tributaria
Alternativas	(a) Término de firmeza de la declaración de renta (art. 632 E.T. y art. 46 Ley 962 de 2005). (b) 10 años de la regla comercial (art. 28 Ley 962 de 2005, art. 60 C. Co.)
Criterio	Cumplir la obligación más larga que aplica a una SAS comerciante
Elección	Plazo por clase: soportes tributarios hasta la firmeza de la declaración (art. 46 Ley 962, modificado por art. 304 Ley 1819); libros y papeles mercantiles 10 años (art. 28 Ley 962). Diseño: 2 años en línea y S3 Object Lock hasta 10 años para soportes aplicables; validar matriz con asesor tributario
Sacrificio	Almacenamiento en S3 después del año 5 (bajo); no se puede borrar antes ni con permisos de administrador, salvo las supresiones de datos personales, que se manejan por seudonimización
Fuente	Estatuto Tributario art. 632; Ley 962 de 2005 arts. 28 y 46; Ley 1819 de 2016 art. 304

F-22. Recomendación final: Escenario A

Campo	Contenido
Contexto	Hay que recomendar un escenario y decir si conviene cambiar en algún año
Alternativas	A (todo en AWS + estadio local) o B (aplicación en Bogotá + base en AWS + estadio local)
Criterio	Costo, riesgo, RPO/RTO, pico, operación y crecimiento
Elección	A: COP 469,7 M de caja en 5 años frente a 697,1 M; latencia ≈ 22 ms frente a ≈ 353 ms; sin depender del enlace Bogotá–AWS; sigue cabiendo con doble preventa y doble cola, y B no en el año 5
Sacrificio	Toda la operación depende de AWS; los datos quedan fuera de Colombia (también en B)
Cuándo cambiar	No hay un año en que B resulte mejor. Se revisaría si una norma exigiera los datos en Colombia o si AWS abriera una región en Colombia con DynamoDB (A se mudaría a esa región)

F-23. 12 terminales portátiles en vez de 24

Campo	Contenido
Contexto	El diseño del estadio pedía 2 lectores portátiles por puerta; la referencia disponible (Zebra TC21, COP 1.973.336) haría que 24 unidades costaran 47,4 M
Alternativas	(a) 24 portátiles. (b) 12, uno por puerta. (c) Terminales genéricos importados (≈ COP 730.000, sin soporte local)
Criterio	Cubrir accesibilidad y un carril caído por puerta con soporte de marca
Elección	(b): COP 23,7 M
Sacrificio	Si caen dos carriles de una misma puerta a la vez, solo uno se cubre con portátil; la gente pasa por los otros carriles de esa puerta

F-24. Fondo de reposición del hardware del estadio

Campo	Contenido
Contexto	El enunciado pide garantía y soporte por 5 años o su renovación; los precios de tienda traen 1 año de garantía y no hay cotización de soporte extendido
Alternativas	(a) Suponer soporte de 5 años sin precio. (b) Reservar cada año un porcentaje del hardware para reponer equipos. (c) Cotizar soporte del fabricante (no disponible hoy)
Criterio	No omitir un costo que podría cambiar la conclusión (−0,5)
Elección	(b): 10 % anual del hardware del estadio desde el año 2 (COP 15,6 M/año), más los 3 nodos de repuesto ya incluidos
Sacrificio	Es un supuesto del grupo, no una cotización; con una cotización real del fabricante se reemplaza

F-25. Escenario B sin caché en espera en us-east-2

Campo	Contenido
Contexto	Con todo costeado, B costaba ≈ COP 300,1 M en el año 1 y ≈ COP 132,2 M en el año 5: no cabía ni con la reserva en el año 5
Alternativas	(a) Declarar B no viable. (b) Quitar la caché en espera de us-east-2 (−10,5 M/año) y usar la reserva en el año 1
Criterio	Presentar dos escenarios viables, con el sacrificio dicho
Elección	(b): B queda en 289,6 M (año 1, con reserva) y 121,7 M (año 5)
Sacrificio	Si se pierde la región, B pierde el estado de la cola virtual y readmite a los compradores por sorteo (runbook 2c de Miguel ya lo prevé); A conserva la cola

Anexo de concurrencia: cuatro escenarios sin doble venta

Autor: David · **Fecha:** 2026-10-29 · **Rúbrica:** Anexo de concurrencia (3 puntos) **Estado:** escenarios 1 y 2 completos (semana 1); escenarios 3 y 4 en borrador para la semana 2. Pseudocódigo completo de las transacciones: `david/concurrencia.md` §3–§5. Claves: `jacobo/modelo_datos.md` §3.2. Referencias `[R-xx]` en `jacobo/referencias.md`; `[D-xx]` en `david/concurrencia.md` §9.

Notación: silla `E1234#S07#R12 / S024`, abreviada **S024**. `t` en segundos; los estados de la silla son `A` (disponible), `L` (bloqueada), `S` (vendida).

Escenario 1: dos clientes piden la misma silla en el mismo milisegundo

Respuesta corta: solo una transacción gana. DynamoDB las ejecuta como si una terminara antes de que empiece la otra; la segunda se cancela por conflicto o encuentra la silla ya bloqueada, y en ambos casos no se escribe nada [R-04].

Transacción de cada cliente (fase 1):

```
TransactWriteItems  ClientRequestToken = H(oid, "LOCK")
  Update bya_seats  E1234#S07#R12 / S024
    IF  attribute_exists(pk) AND ( st='A' OR (st='L' AND lx < :vencido_antes_de) )
    SET st='L', lx=:lx, lo=:oid   ADD v :uno
  Put    bya_orders  ORD#<oid> / ORDER   (st='PENDING', ...)   IF attribute_not_exists(pk)
  Update bya_counters LIM#E1234#U<uid> / P   SET n = if_not_exists(n,:cero) + :k   IF
attribute_not_exists(n) OR n <= :max_menos_k
```

Línea de tiempo. Ana (orden `O-ANA`) y Beto (orden `O-BETO`), S024 en `A`, `v = 0`.

t	Sesión de Ana	Sesión de Beto	S024
0,000	Envía su transacción	Envía su transacción	`A`, v=0
0,003	DynamoDB evalúa su condición: `st='A'` se cumple. Escribe `L`, `lo=O-ANA`, `lx=480`	Su transacción toca el mismo ítem mientras la de Ana está en curso	`L` (O-ANA), v=1
0,005	Recibe éxito; ve su temporizador de 8 min	Recibe `TransactionCanceledException`, motivo `TransactionConflict` en la acción de S024 [R-04, D-02]	`L` (O-ANA)
0,040	Pasa a pagar	Espera aleatoria (*backoff*) y reintenta con el **mismo** token	`L` (O-ANA)
0,045	—	La condición se evalúa ahora contra `st='L'` con `lx` en el futuro: **no se cumple**	`L` (O-ANA)
0,047	—	Recibe `ConditionalCheckFailed` en S024. La API responde 409 “silla no disponible” y no reintenta	`L` (O-ANA)

Variante: si Beto llega cuando la transacción de Ana ya terminó, salta directo a la fila 0,045. El resultado no cambia.

Por qué no hay doble venta:

1. El aislamiento entre transacciones, y entre transacciones y escrituras sueltas, es **serializable** [R-04]. No existe un orden de ejecución en el que ambas condiciones se cumplan, porque la primera que escribe cambia `st` a `L`.
2. La transacción de Beto es **todo o nada**: su orden `O-BETO` no se crea y su cupo `LIM#` no se descuenta [R-04, D-02].

3. Si Ana pedía 3 sillas y solo S024 estaba en disputa, igual se cancela **toda** su transacción si pierde; nunca queda con 2 de 3 sillas bloqueadas. `CancellationReasons` dice cuál silla falló, en el orden de las acciones [D-02], y la API se lo muestra al cliente.

Escenario 2: el bloqueo vence mientras el cliente paga

Respuesta corta: si nadie tomó la silla, la compra se confirma aunque `lx` haya pasado. Si otro la tomó después del vencimiento, la confirmación falla por condición, la orden pasa a `FULFILL_FAIL` y el pago se reembolsa. En ningún caso dos órdenes terminan con la misma silla.

Transacción de confirmación (fase 3), parte que decide:

```
TransactWriteItems  ClientRequestToken = H(oid, "CONFIRM")
  Update bya_seats   S024             IF st='L' AND lo=:oid          SET st='S', so=:oid, tk=:tid
ADD v :uno
  Put    bya_tickets TKT#<tid> / T    IF attribute_not_exists(pk)
  Update bya_orders  ORD#<oid>/ORDER  IF st='PENDING'                SET st='PAID', ...
  Update bya_orders  ORD#<oid>/PAY#01 IF st IN (:initiated,:unknown) SET st='APPROVED', gtx=:gtx
REMOVE g3pk, g3sk
  Update bya_counters CNT#E1234#S07 / SH#<hash(oid) mod 8>           ADD sold :uno
  Put    bya_orders  ORD#<oid>/INV#1  IF attribute_not_exists(pk)
```

2a. Venció, pero nadie la tomó (se confirma)

t	Sesión de Ana	Otros	S024
0	Bloqueo con éxito, `lx = 480`	—	`L` (O-ANA)
60	Registra `PAY#01` en `INITIATED`; abre la pasarela	—	`L` (O-ANA)
481	Sigue en la pasarela	Nadie pidió S024	`L` (O-ANA), vencida
495	La pasarela aprueba; se ejecuta la confirmación	—	`L` (O-ANA)
495	`st='L' AND lo='O-ANA'` **se cumple**: no se exige que `lx` siga vigente	—	`S` (O-ANA)

2b. Venció y otro la tomó (reembolso)

t	Sesión de Ana	Sesión de Beto	S024
0	Bloqueo con éxito, `lx = 480`	—	`L` (O-ANA)
60	Registra `PAY#01` en `INITIATED`; abre la pasarela	—	`L` (O-ANA)
483	Sigue en la pasarela	Pide S024. `lx = 480 < now − skew = 481`: la condición de bloqueo **se cumple**	`L` (O-BETO)
495	La pasarela aprueba; se ejecuta la confirmación	Pagando	`L` (O-BETO)
495	`lo='O-ANA'` **no se cumple**: `ConditionalCheckFailed`. Se cancela **toda** la transacción: ni boleta, ni `PAID`, ni contador	—	`L` (O-BETO)
496	Orden → `FULFILL_FAIL`; `PAY#01` → `APPROVED` (queda en `gsi_pendientes`); reembolso a SQS; aviso a Ana	—	`L` (O-BETO)
530	—	Confirma con normalidad	`S` (O-BETO)

Por qué no hay doble venta ni dinero perdido:

1. La silla solo pasa a `S` en una transacción cuya condición nombra al dueño actual (`lo`). Para que Beto sea dueño, su propio bloqueo tuvo que escribir `lo='O-BETO'`, y esa escritura es serializable con la confirmación de Ana [R-04].
2. El vencimiento se evalúa **dentro de la condición** de Beto, no con TTL. El TTL puede tardar días y no se usa para la lógica [R-08].
3. El dinero de Ana no se pierde: el pago queda `APPROVED` con la orden en `FULFILL_FAIL`, y `gsi_pendientes` lo mantiene visible para el reconciliador hasta que el reembolso termina.
4. Para que 2b sea raro, la sesión de la pasarela vence antes que el bloqueo (7 de los 8 minutos).

Escenario 3: falla a mitad de la compra

Respuesta corta: una falla puede dejar la compra *sin terminar*, nunca *a medias*. Las dos transacciones (bloqueo y confirmación) son todo o nada [R-04]; el cobro, que no es transaccional, se protege con un registro previo (`PAY#nn`), una llave de idempotencia con la pasarela y un reconciliador. En ningún caso se cobra sin que quede rastro, ni se vende una silla dos veces.

Punto de falla	Qué queda en la base	Quién lo resuelve	Mecanismo
3a. Durante la transacción de bloqueo o confirmación	Nada o todo	DynamoDB	Atomicidad [R-04]
3b. Respuesta perdida (*timeout*)	No se sabe si se aplicó	El servidor	Reintento con el mismo `ClientRequestToken` (10 min); luego lectura fuerte
3c. Servidor cae tras cobrar, antes de confirmar	`PAY#01` `INITIATED`, orden `PENDING`	Webhook, SQS o reconciliador	Confirmación idempotente; DLQ [D-04]
3d. La pasarela no responde	`PAY#01` `UNKNOWN`	Reconciliador	Consulta con la llave `<oid>#<nn>`

3a. Falla durante la transacción

Si el servidor cae, o una de las condiciones falla, DynamoDB cancela toda la transacción y no deja efectos parciales; si se cancela, `CancellationReasons` dice cuál acción falló, en el mismo orden de las acciones [D-02]. La atomicidad vale frente a operaciones individuales; las lecturas con `Query` o `Scan` son *read committed* y pueden ver el estado antes o después, pero nunca la mitad de la transacción [R-04].

3b. Respuesta perdida

t	Servidor	Base	Qué pasa
0	Envía `TransactWriteItems` (token `T1`)	Aplica la transacción	—
1	La respuesta se pierde; el cliente HTTP da *timeout*	Sillas en `L`	El servidor **no sabe** si se aplicó
2	Reintenta con el **mismo** `T1`	Reconoce el token	Devuelve éxito sin repetir
3	Si pasaron más de 10 min o cambió algún parámetro	`IdempotentParameterMismatch` o ejecuta de nuevo	Lectura fuerte de `ORDER` y de las sillas

```
funcion reintentarBloqueo(oid, plan):
    token = H(oid, "LOCK")                          # mismo token en cada reintento
    repetir hasta 3 veces con espera 50 ms x 2^n + jitter:
        intentar TransactWriteItems(plan, ClientRequestToken = token): devolver OK
        capturar TransactionInProgress, InternalServerError, timeout: continuar
        capturar IdempotentParameterMismatch: leer ORDER (fuerte); decidir segun estado
        capturar TransactionCanceledException e:
            si e.motivo(Put ORDER) == ConditionalCheckFailed y ORDER (fuerte) existe con este uid:
                devolver OK                          # el intento anterior ya se aplicó
            devolver CONFLICTO                       # la silla es de otro; no se reintenta
    leer ORDER (fuerte): si st='PENDING' con sillas en L de oid -> OK; si no -> ERROR
```

El token protege 10 minutos [D-02]; el bloqueo dura 8, así que el reintento cae dentro de la ventana. La confirmación tiene además su propia idempotencia por condiciones (`st='PENDING'`, `attribute_not_exists`), que sigue funcionando después de esos 10 minutos.

3c. El servidor cae después de cobrar

t	Servidor A	Pasarela / webhook	Base
0	Escribe `PAY#01 INITIATED` y llama a la pasarela con la llave `<oid>#01`	Procesa el cobro	Orden `PENDING`, sillas `L`
40	**Cae** antes de confirmar	Aprueba; envía webhook	Igual
41	—	El webhook llega a un servidor sano que lo encola en SQS	Igual
42	Servidor B toma el mensaje y ejecuta la confirmación de `concurrencia.md` §5	—	Orden `PAID`, sillas `S`, boletas
43	—	Reintento duplicado del webhook	La confirmación encuentra `st<>'PENDING'`: no hace nada (idempotente)

Si el webhook se pierde, el reconciliador (índice disperso `gsi_pendientes`, cada minuto, pagos de más de 5 minutos) consulta la pasarela con la llave `<oid>#01`, ve el cobro aprobado y confirma. Si el mensaje falla varias veces, va a la DLQ tras `maxReceiveCount` intentos y salta una alarma [D-04].

Si pasaron más de 8 minutos y otro tomó la silla, aplica el escenario 2b: reembolso, no doble venta.

3d. La pasarela no responde

t	Servidor	Pasarela	Base
0	`PAY#01 INITIATED`; llama	No contesta (*timeout*)	`INITIATED`
10	Marca `PAY#01 → UNKNOWN` (no `DECLINED`)	Puede haber cobrado o no	`UNKNOWN`, sillas `L`
60	Reconciliador consulta con la llave `<oid>#01`	Responde: aprobado, rechazado o inexistente	—
61	Aprobado: confirma. Rechazado/inexistente: libera las sillas	—	`PAID` o `ABANDONED`

Regla: nunca se libera una silla ni se rechaza una compra mientras el pago esté en `UNKNOWN`: puede haberse cobrado. La llave de idempotencia evita que el reintento cobre dos veces.

Por qué no hay doble cobro: (1) la llave `<oid>#nn` se envía a la pasarela en cada intento; (2) `PAY#nn` existe *antes* de llamar; (3) un segundo intento del usuario crea `PAY#02` solo si `PAY#01` está `DECLINED`.

Escenario 4: conmutación (falla de servidor, zona o región)

Respuesta corta: servidor y zona no pierden nada y no exigen conmutar la base. La región sí: la base puede perder los últimos segundos (MREC es asíncrona), pero el dinero no: las sillas de todo pago reciente se congelan antes de reabrir y se concilian después con la pasarela. Detalle en `alta_disponibilidad.md`.

Falla	RPO	Qué hacer	Fuente
Servidor de aplicación	0 (el estado vive en la base)	Otro servidor retoma; los reintentos son idempotentes	Escenario 3
Zona de disponibilidad	0	Nada: DynamoDB replica en 3 zonas	[R-01], [D-11]
Región completa	> 0 en la base (segundos); 0 en pagos por proceso	Cerrar admisión, mover tráfico, congelar sillas en duda, reabrir; conciliar después	[D-07], D-2

4a. Línea de tiempo: pérdida de la región principal en plena preventa

t	us-east-1 (activa)	us-east-2 (réplica)	Pasarela	Acción
0	Opera; las ventas escriben	Recibe replicación (típicamente ≤ 1 s) [D-07]	Cobra	—
100 s	**Falla**	Último dato replicado: T	Sigue cobrando a quien ya estaba pagando	—
+1,5 a +5 min	—	—	—	Route 53 (3 revisiones × 30 s) y alarma compuesta; confirmación humana. **Detección, fuera del RTO** [D-22, D-10]
0 (declaración)	—	—	—	**Declaración.** Se registran T y el último `ReplicationLatency`
+1 a +2 min	—	Cierra admisión	—	Congelar escrituras
+2 a +5 min	—	Recibe el tráfico; aplicación lista en paralelo	—	Cambiar tráfico
+3 a +7 min	—	Congela las sillas de todo intento de pago de `bya_ledger` desde `T − W`	Consulta cada pago por su id	**Congelar sillas en duda**
+4 a +8 min	—	**Reabre por tramos**	—	**Fin del RTO**
Después	—	`confirmarDesastre` por cada pago; libera las congeladas no vendidas; reconstruye contadores	Lista pagos	Conciliar con la venta abierta

Peor caso, con todo en secuencia: 11 minutos desde la declaración (`alta_disponibilidad.md` §3.3). Los tiempos son **estimados** y solo un simulacro los confirma.

4b. Por qué no hay doble venta en el cambio

Dos regiones escribiendo la misma silla podrían venderla dos veces, porque la condición se evalúa contra la versión local y gana la última escritura [D-07]. Por eso escribe **una sola región**. Y como el profesor acepta reabrir antes de terminar la conciliación si las sillas en duda siguen congeladas (respuestas 3b y 6), toda silla que pudo venderse en us-east-1 sin llegar a la réplica queda bloqueada a nombre del incidente. La lista sale del libro de pagos `bya_ledger` (D-2b), que se escribe antes de cobrar y no se pierde con la región:

```
funcion conmutar(T):
    cerrarAdmision()
    enrutarTrafico(regionAlterna)                # en paralelo: esperar aplicacionLista()
    W = 8 min + 2 min + ultimo ReplicationLatency
    si no congelarSillasEnDuda(T, W):            # alta_disponibilidad.md §5.0: lee bya_ledger (MRSC)
        mantener la venta cerrada y reintentar    # sin el libro no se sabe qué está en duda
    reabrirGradual()                             # FIN DEL RTO
    para cada intento en bya_ledger(desde = T - W) ordenado por fecha:      # pago pedido por su id
        confirmarDesastre(pago)                  # §5.2; acepta sillas congeladas por el incidente
    liberarCongeladasNoVendidas()
    reconstruirContadores()
```

Dos bloqueos a la vez sobre la misma silla (prohibido, respuesta 5) tampoco ocurre. Un bloqueo que no llegó a la réplica queda anulado: su orden no existe en us-east-2, y sin orden no se puede pagar. Si el cliente ya había

empezado a pagar, su intento está en el libro y la silla queda congelada.

4c. Qué se pierde y qué no

Elemento	Se puede perder	Cómo se cubre
Bloqueos de sillas de los últimos segundos	Sí	Las sillas vuelven a `A`; el cliente reintenta
Órdenes `PENDING` sin cobrar	Sí	El cliente vuelve a comprar
Compras **cobradas**	Segundos de base	Sus sillas quedan congeladas y se reaplican desde la pasarela; sin metadatos, se reembolsan
Boletas ya entregadas	Posible	`confirmarDesastre` las recrea con `TKT#UUIDv5(oid, silla)`, el mismo identificador

---

Referencias. [R-xx]: `jacobo/referencias.md`. [D-01] a [D-04]: `concurrencia.md` §9. [D-05], [D-06]: `estadio.md`. [D-07] a [D-14], [D-18] y [D-22]: `alta_disponibilidad.md` §9.

Conmutación y alta disponibilidad

Autor: David · **Fecha:** 2026-11-03, actualizado el 2026-11-04 con las respuestas del profesor (`respuestas_profesor.md`) · **Rúbrica:** Estudio de conmutación y alta disponibilidad (3 puntos) **Insumos:** `jacobo/arquitectura.md` §4.1, `miguel/recuperacion_licenciamiento_carta.md` (tabla de RTO y runbooks), `miguel/respuesta_a_david.md`. Referencias `[R-xx]` en `jacobo/referencias.md` (citas de Jacobo que no abrí yo); `[D-xx]` al final de este documento (páginas que abrí yo el 2026-11-03).

> **Cambio del 2026-11-04 (respuestas 3 y 6 del profesor):** ya no se cierra toda la venta al perder la región. Se **congelan solo las sillas en duda** (§5.0), se reabre, y la conciliación termina después con esas sillas congeladas. El RTO termina en la reapertura y baja a **4 a 11 minutos** desde la declaración (§3.3).
>
> **Idea central:** la base no tiene un "servidor principal" que conmutar. Dentro de la región DynamoDB replica en 3 zonas y no hay nada que hacer. Lo que sí se conmuta, y es lo que cuesta tiempo, es **el tráfico y la aplicación** hacia otra región, y ahí el RPO 0 de los **pagos** se logra con un **proceso** (conciliar contra la pasarela), no con la réplica.

---

1. Resumen

Falla	RPO de la base	RPO de pagos	RTO	Mecanismo	Fuente
Servidor de aplicación	0	0	Lo que tarde el escalado	El estado vive en la base; la confirmación es idempotente	`concurrencia.md` §5
Zona de disponibilidad (para el profesor es la falla del "servidor principal", no del sitio)	0	0	Sin conmutación de la base; la aplicación corre en varias zonas	Replicación en 3 zonas, SLA 99,99 %	[D-11, D-12]
Región completa (= pérdida del sitio en la nube)	El atraso de replicación: típicamente ≤ 1 s, sin SLA (§3.1)	0, con las sillas en duda congeladas hasta conciliar	**4 a 11 min desde la declaración** hasta aceptar compras (§3.3); detección y declaración aparte, 2 a 5 min	Réplica en us-east-2 + congelamiento de sillas en duda + conciliación contra la pasarela	[D-07, D-08]
Data center de Bogotá (= pérdida del sitio en el Escenario B)	0 (la base no está allí)	0	3 a 11 min (Miguel)	Aplicación en espera en AWS; runbook 2c	`miguel/` runbook 2c

2. Cómo funciona la alta disponibilidad elegida

2.1 Dentro de la región: replicación en 3 zonas

AWS dice que DynamoDB "replica automáticamente tus datos en tres zonas de disponibilidad para dar alta durabilidad y un SLA de disponibilidad de 99,99 %" [D-11] (traducción mía). La escritura se confirma cuando está replicada, por eso **un fallo de zona no pierde datos** (RPO 0) y **no hay conmutación que gestionar**: no hay versiones ni ventanas de mantenimiento [D-11].

2.2 Entre regiones: tabla global MREC con una sola región que escribe

Decisión	Elección	Por qué
Tipo de tabla global	**MREC** (consistencia eventual entre regiones)	La compra usa `TransactWriteItems`. MRSC (RPO 0 entre regiones) "no soporta operaciones de transacción y devolverá un error si se invocan", tampoco TTL, y exige exactamente tres regiones [D-07]
Modo de escritura	**Una sola región escribe** (*write to one Region*): us-east-1 activa, us-east-2 en espera	AWS describe este modo como el que exige "un mecanismo para enrutar las escrituras a la región activa" [D-09]. Evita que dos regiones escriban la misma silla (ver abajo)
Regiones	us-east-1 principal, us-east-2 alterna	Decisión D-3 de Jacobo; MRSC no existe en sa-east-1 [R-17]

Qué significa MREC para nuestra regla de oro (todo textual de [D-07], traducción mía):

1. **Transacciones.** "Son solo atómicas dentro de la región donde se invocó la operación" y no se replican como unidad: en otra réplica "solo algunas de las escrituras de una transacción podrían verse". Una réplica puede mostrar una compra **a medias**.
2. **Escrituras condicionales.** "Evalúan la condición contra la versión del ítem en la región". Si dos regiones escribieran, cada una vería su propia versión y **ambas podrían vender la misma silla**.
3. **Conflictos.** Gana "la última escritura" por ítem, según una marca de tiempo interna, y esos conflictos no quedan en CloudWatch ni CloudTrail [D-07, D-08].
4. **Lecturas fuertes.** Devuelven lo último "si el ítem se actualizó por última vez en la región donde ocurre la lectura, pero pueden devolver datos viejos si se actualizó en otra región" [D-07].

De los puntos 1 y 2 sale la regla: **solo una región escribe**. De los puntos 1 y 4 sale la necesidad de la conciliación: tras perder la región, lo que la réplica muestra **no es de fiar** para las últimas ventas.

SLA. El de 99,999 % exige que **todas** las tablas de la región sean globales y que se intente la conmutación en un problema regional; si no, aplica el de 99,99 % [D-12]. Con ambas cifras se cumple el mínimo del enunciado (Miguel hace la misma observación).

3. Cuánto tarda la conmutación

3.1 Qué publica AWS (y qué no)

Dato	Valor publicado	Fuente
Conmutación de la **base**	“No hay conmutaciones complicadas o demoradas ni tiempo de inactividad de la base al conmutar una aplicación entre regiones” (la base ya está en las dos)	[D-11]
Retraso de replicación MREC	“Típicamente dentro de un segundo o menos”; el RPO es igual al retraso, “usualmente unos segundos”; **no hay SLA** sobre esa latencia	[D-07, D-08]
Replicación bajo carga	Capital One midió ≈ 1 s en operación normal y que “bajo carga fuerte la latencia aumentó” (sin cifra)	[D-14]
Cambio de tráfico con DNS	TTL de 60 s es típico; no todo el software respeta el TTL ni los cachés intermedios	[D-09]
Cambio de tráfico, extremo a extremo	Con ARC: “segundos a pocos minutos, según la propagación del TTL de DNS”. Solo con DNS: “varios minutos”	[D-10]
Revisión de salud de Route 53	Cada 10 o cada 30 s; el tiempo total depende del umbral de fallos y **AWS no da una fórmula**	[D-13]
RTO absoluto de la conmutación de una región de DynamoDB	**AWS no publica una cifra**	[D-08, D-10]

Atraso de replicación que asumimos (condición a del profesor). Típicamente 1 s o menos [D-07]; Capital One midió ≈ 1 s y vio que crece bajo carga [D-14]; AWS no da SLA [D-08]. Diseñamos para **hasta 60 s**: una alarma salta si `ReplicationLatency` pasa de 60 s durante la preventa, y en ese caso el coordinador sabe que la ventana de §5.0 debe crecer en lo mismo. **La seguridad no depende de esa cifra**: las sillas en duda salen de la pasarela, no de la réplica, y la ventana se fija como 8 minutos de bloqueo + 2 de margen + el último atraso medido.

Consecuencia honesta: el único dato con número es el de enrutamiento (segundos a minutos). Todo el resto del RTO (congelar, levantar la aplicación, conciliar) son **tiempos de nuestro propio procedimiento**, y solo un simulacro los confirma.

3.2 Criterio para declarar la falla

El enunciado mide el RTO “desde que se declara la falla” (hallazgo H-2 de Miguel), así que el criterio debe estar acordado **antes** de la preventa. Propuesta, basada en lo que AWS recomienda [D-10]:

- **Alarma compuesta** de CloudWatch con `SystemErrors` de DynamoDB, `ReplicationLatency` y el porcentaje de errores 5xx de la API. AWS da como ejemplo una alarma que se activa cuando los 5xx “superan 5 % durante tres periodos consecutivos de 1 minuto” [D-10].
- **Confirmación humana:** el coordinador del incidente y el DBA de guardia revisan el estado de AWS (AWS Health) y **declaran**. Es una decisión manual a propósito: declarar congela sillas y mueve todo el tráfico de región.
- **Revisión de salud de Route 53** contra el punto de compra de us-east-1: cada 30 s y falla tras 3 revisiones seguidas, que son los valores por defecto (umbral entre 1 y 10; intervalo de 10 o 30 s) [D-22]. Con eso la detección automática tarda unos **90 s**; con el intervalo rápido de 10 s, unos 30 s, con costo adicional [D-13].
- Se anotan la hora **T** de la última escritura visible y la última `ReplicationLatency` (paso 1 del runbook 2b).

Tiempo de detección y declaración, reportado aparte del RTO (respuesta 6 del profesor):

Etapa	Tiempo	Fuente
Route 53 marca la falla (3 revisiones × 30 s)	≈ 1,5 min	[D-22]
La alarma compuesta de CloudWatch se activa (3 periodos de 1 min)	≈ 3 min, en paralelo	[D-10]
El coordinador y el DBA confirman en AWS Health y declaran	0 a 2 min	Supuesto
Detección y declaración	**≈ 2 a 5 min**	Se mide en el simulacro

Criterio de declaración: alarma compuesta activa **y** Route 53 en falla **y** confirmación humana. Es manual a propósito: congelar y conmutar no se hace por un falso positivo.

3.3 RTO nuevo: la conciliación sale del camino crítico

El profesor dice que el RTO termina “hasta que la plataforma acepta compras otra vez con datos correctos”, y que “la conciliación de pagos pendientes puede terminar después, siempre que esas sillas sigan congeladas” (respuesta 6). Eso saca la conciliación y la reconstrucción de contadores del RTO.

Paso	Minutos	Fuente u observación
1. Cerrar la admisión y no enrutar a us-east-1	1 a 2	Miguel
2. Cambiar el tráfico a us-east-2 (la gente ve la sala de espera)	1 a 3	“Segundos a pocos minutos” con ARC, “varios minutos” solo con DNS [D-10]; vale con TTL de 60 s y el cambio preparado
3. Aplicación en us-east-2 lista (en paralelo con el paso 2)	0 a 3	Miguel; réplica pre-calentada automáticamente [D-18]
4. **Congelar las sillas en duda** (§5.0)	1 a 2	Supuesto: leer `bya_ledger` por minuto, consultar por `gid` en Mercado Pago y hacer unos pocos miles de escrituras
5. Reabrir por tramos	1	Miguel
RTO (pasos 2 y 3 en paralelo)	**4 a 8**	Suma de mínimos y máximos
RTO (todo en secuencia)	**4 a 11**	Peor caso
Después del RTO: conciliar (§5.1–5.3) y reconstruir contadores (§5.4)	4 a 7	No cuenta en el RTO: las sillas en duda siguen congeladas

Cumplimiento de la condición c del profesor: el peor caso estimado (11 min) queda **por debajo de 15**, con 4 minutos de margen. Si el simulacro midiera más de 15, la diferencia se cierra con lo que más pesa: automatizar el cambio de tráfico con Application Recovery Controller en vez de DNS manual [D-10], cuyo costo debe cotizar Juan Pablo. Ningún tiempo de esta tabla está medido.

Total desde la falla, para informarlo completo: 2 a 5 min de detección y declaración más 4 a 11 de RTO, es decir **6 a 16 minutos**.

4. Por qué ninguna orden confirmada se pierde (RPO 0)

Pregunta del jurado	Respuesta
¿Se pierde una orden si cae un servidor?	No. La orden y el pago están en la base y la confirmación es idempotente (`concurrencia.md` §5–§6)
¿Y si cae una zona?	No. Se confirma tras replicar en 3 zonas [D-11]
¿Y si cae una región?	La **base** puede perder los últimos segundos (RPO = retraso de replicación [D-07]). Pero el **dinero** no se pierde: la pasarela guarda cada cobro, las sillas de esos cobros se congelan antes de reabrir y se concilian después
¿Puede entonces venderse dos veces una silla?	No. Toda silla que pudo venderse en us-east-1 sin llegar a la réplica tiene un cobro en la pasarela, y esa silla queda congelada hasta conciliarla. Además, solo escribe una región
¿Pueden dos personas tener la misma silla bloqueada a la vez?	No. Un bloqueo que no llegó a la réplica queda anulado: su orden no existe en us-east-2, así que ese cliente no puede pagar con él y debe volver a elegir. Si ya había empezado a pagar, su cobro está en la pasarela y la silla queda congelada (§5.0)

La afirmación “RPO 0 de pagos” depende de la pasarela y del libro de pagos. **Mercado Pago** (D-11) cumple lo que se probó en el sandbox el 2026-11-04 (`pasarelas.md` §7): `metadata` guarda y devuelve las sillas, repetir con la misma `X-Idempotency-Key` devuelve el mismo pago y un pago se puede pedir por su `id`. **La búsqueda por fechas no funcionó en la corrida de sandbox**, y de ahí salía la lista de sillas en duda. Por eso el grupo aprobó **D-2b** el 2026-11-04: esa lista sale de `bya_ledger` (MRSC, RPO 0), que se escribe antes de cobrar (§5.0). Es la condición 1 y 2 de Miguel.

5. Modo desastre: confirmar un pago cuando la orden o la silla se perdieron

Este es el paso 4 del runbook 2b de Miguel. **No sirve la confirmación normal** (`concurrencia.md` §5), por tres razones que salen de §2.2:

Por qué falla la normal	Consecuencia
Lee `orden.sil` de `ORDER`, que puede no existir en la réplica	No sabe qué sillas confirmar
Exige `st='L' AND lo=:oid`	En la réplica la silla puede estar `A` (el bloqueo no alcanzó a replicarse) y la condición falla
Cuenta con que la compra se aplicó completa o nada	Una réplica puede mostrar una compra **a medias** [D-07]: `ORDER` en `PAID` y la silla todavía en `L`

5.0 Congelar las sillas en duda (respuesta 3b del profesor)

Qué silla está en duda. Una silla solo puede estar vendida en us-east-1 sin que la réplica lo sepa si se intentó pagarla: para vender, la compra pasa siempre por un cobro, y antes de cobrar se escribe el intento en el libro de pagos (`concurrencia.md` §4). Por eso:

> **Sillas en duda** = las sillas de **todo intento de pago registrado en `bya_ledger` desde `T − W`**, salvo los que Mercado Pago dé por rechazados (`rejected`) o cancelados. `W` = 8 min de bloqueo + 2 min de margen + el último `ReplicationLatency` medido.

Un pago creado antes de `T − W` corresponde a un bloqueo que ya venció en us-east-1. Si se aprueba después, se trata como en el escenario 2b del anexo: si la silla sigue libre se confirma; si otro la tomó, se reembolsa. Es lo mismo que pasaría sin desastre.

Estados confirmados en el sandbox (2026-11-04): aprobado `approved`, en proceso `in_process` y rechazado `rejected` (`pasarelas.md` §7). La regla de abajo usa esos nombres.

Fuente de la lista: el libro de pagos (D-2b, aprobada por el grupo el 2026-11-04). En el sandbox, `/v1/payments/search` no listó pagos por fechas ni 14 minutos después; solo los encontró filtrando por `id` (`pasarelas.md` §7). Por eso la lista de sillas en duda **no sale de Mercado Pago, sino de `bya_ledger`**, el libro de pagos de Jacobo (tabla global **MRSC**, `jacobo/modelo_datos.md` §3.9). Se escribe **antes** de cobrar, con el `oid` y las

sillas, y se replica de forma sincrónica a la otra región antes de responder, así que sobrevive a la pérdida de la región con RPO 0 [R-17]. Para que sirva así:

1. **Listar por minuto.** Con la clave actual (`pk = LED#<oid>`) habría que recorrer la tabla entera. Propuesta a Jacobo: `pk = LED#<minuto>#<fragmento 0..7>`, `sk = <oid>#<nn>`, con el fragmento sacado del `oid` para no calentar una partición. **La clave la define Jacobo**; el pseudocódigo de abajo usa esta propuesta.
2. **Guardar el `id` del pago** de Mercado Pago apenas responde (paso `INTENT → CHARGED` de su diseño). Con el `id`, el estado se consulta pidiendo el pago por su id, que sí funcionó en el sandbox, y de paso se leen las sillas de su `metadata`.
3. **Un intento sin `id`** (el servidor cayó entre cobrar y guardar) se congela igual y se resuelve con la notificación de Mercado Pago, que reintenta cada 15 minutos hasta recibir respuesta [P-04 en `pasarelas.md`]. Cuando el tráfico ya está en us-east-2, la notificación llega allí.

La lista que da el libro es **más amplia** que la de la pasarela: incluye intentos que nunca llegaron a cobrarse. Es más segura, aunque congela algunas sillas de más hasta que se aclare su estado. El costo es una escritura sincrónica entre regiones en cada pago, cuya latencia nadie ha medido (Jacobo, §3.9), y la tabla MRSC en tres regiones (Juan Pablo).

Cómo se congela, sin estado nuevo. Cada silla en duda se bloquea a nombre del incidente, con un vencimiento lejano. La condición normal de bloqueo (`st = 'A' OR (st = 'L' AND lx < :vencido_antes_de)`) ya rechaza esa silla para cualquier otro cliente:

```
funcion congelarSillasEnDuda(T, W, incidente):
    intentos = para cada minuto m de T-W a NOW, para cada fragmento f de 0 a 7:
                   Query bya_ledger  pk = "LED#" + m + "#" + f   (lectura fuerte, MRSC)   # clave:
propuesta a Jacobo
    si la lectura falla: NO reabrir; seguir con las ventas cerradas y reintentar        # respaldo
seguro
    para cada intento i:
        si i.gid existe:                                          # id del pago en Mercado Pago
            p = MercadoPago GET /v1/payments/{i.gid}              # por id sí funciona (sandbox)
            si p.status in (rejected, cancelled): continuar     # no está en duda
        # sin gid: el pago puede o no haberse creado -> se congela igual
        para cada silla s de i.sil:
            Update bya_seats Key de s
                IF  st <> 'S'                                      # una vendida ya no se puede comprar
                SET st = 'L', lo = :conc, lx = :lejos ADD v :uno   # :conc = "CONC#<incidente>",
:lejos = T + 1 día
    registrar en el incidente cuántas sillas quedaron congeladas
```

- Si el `Update` falla porque la silla está `S`, ya está protegida: no hace falta congelarla.
- Pisar un bloqueo `L` de la misma orden no hace daño: `confirmarDesastre` acepta las sillas congeladas por el incidente (§5.2).
- El mapa mostrará esas sillas como no disponibles. El profesor acepta que el mapa vaya con atraso mientras la validación final sea contra la base (respuesta 5).
- El límite de 10 sillas por usuario (`LIM#`) puede estar desactualizado en la réplica al reabrir. Eso no causa doble venta y se recalcula al final de la conciliación (§5.4).

Cuándo se libera una silla congelada. Según el estado final de su pago:

Estado del pago	Qué se hace con sus sillas
Aprobado	`confirmarDesastre` las vende a esa orden (§5.2)
Rechazado o cancelado	Se liberan
Todavía en proceso (por ejemplo, un pago que tarda en confirmarse)	**Siguen congeladas** hasta que la pasarela dé el estado final. El webhook o el reconciliador decide entonces: aprobado, se confirma; rechazado, se liberan

Liberar es esta escritura, que solo toca sillas que siguen a nombre del incidente:

```
Update bya_seats Key de s  IF st = 'L' AND lo = :conc  SET st = 'A'  REMOVE lx, lo  ADD v :uno
```

5.1 Datos de partida y precondiciones

- **Entrada: los mismos intentos que se usaron para congelar** (§5.0): todo intento de `bya_ledger` desde `T − W`, no solo los últimos 5 minutos. Para cada uno, el pago se pide a Mercado Pago por su `id` (`gid`); si no hay `id`, se espera la notificación de Mercado Pago. Conciliar una ventana más corta que la del congelamiento dejaría sillas congeladas sin revisar, y al liberarlas se podría perder una compra pagada. Cada pago trae `oid`, `nn`, `gtx`, `uid`, `eid`, `sillas[]`, `monto` y `aprobado_en` en su `metadata`. Los aprobados pasan por `confirmarDesastre`; los que siguen en proceso esperan (ver abajo).
- **Precondiciones:** las sillas en duda ya están congeladas (§5.0); la aplicación en us-east-2 apunta a la réplica local; nadie escribe en us-east-1. Las ventas pueden estar **abiertas**: ninguna otra compra puede tocar una silla congelada.
- **Orden de proceso:** por `aprobado_en` ascendente. Si dos cobros aprobados reclaman la misma silla, gana el primero y el otro se reembolsa. Nunca hay doble venta; en el caso raro puede que la silla vaya a quien no la tenía en us-east-1, y por eso el criterio es explícito.

5.2 Pseudocódigo

```
funcion confirmarDesastre(cobro):                    # cobro viene de la pasarela
    ahora = epoch_segundos()
    :vencido_antes_de = ahora - :skew                # las condiciones no admiten aritmética [D-01]

    repetir hasta 3 veces:                           # un choque con otra escritura obliga a replanificar
        # 1. LEER (fuerte, en la réplica local). Las lecturas por lote no necesitan ser transaccionales.
        orden   = GetItem(ORD#oid / ORDER)
        pago    = GetItem(ORD#oid / PAY#nn)
        sillas  = BatchGetItem fuerte de cada silla de cobro.sillas
        tickets = BatchGetItem fuerte de TKT#UUIDv5(oid, silla)
        inv     = GetItem(ORD#oid / INV#1)

        # 2. DECIDIR si todas las sillas son nuestras o están libres
        para cada silla s:
            mia   = (s.st == 'S' y s.so == oid)
            libre = (s.st == 'A')
                 o (s.st == 'L' y (s.lo == oid o s.lo == :conc o s.lx < :vencido_antes_de))
            si no (mia o libre): conflicto = verdadero

        si conflicto:
            # alguien más tiene la silla (S de otra orden o bloqueo vigente de otra)
            planificar ORDER -> 'FULFILL_FAIL' (Put si falta; Update IF st <> 'PAID' si existe)
            planificar PAY   -> 'APPROVED' (Put si falta; Update IF st IN (INITIATED, UNKNOWN))
            ejecutar; encolar reembolso con llave refund#oid; devolver REEMBOLSO

        # 3. PLANIFICAR solo lo que falta (cada acción afirma lo que se leyó)
        acciones = []
        para cada silla s que no es 'mia':
            acciones += Update bya_seats  Key de s
                IF  (st = 'A')
                 OR (st = 'L' AND (lo = :oid OR lo = :conc OR lx < :vencido_antes_de))
                 OR (st = 'S' AND so = :oid)                    # por si otra ejecución ya la tomó
                SET st = 'S', so = :oid, tk = :tid  REMOVE lx, lo  ADD v :uno
        para cada ticket ausente:
```

```
            acciones += Put bya_tickets TKT#tid / T (st='VALID', ...)  IF attribute_not_exists(pk)
        si orden ausente:   acciones += Put    ORDER (st='PAID', sil=cobro.sillas, tot=cobro.monto,
                                                      uid, eid, ttl=:ttl_largo)  IF
attribute_not_exists(pk)
        si orden != 'PAID': acciones += Update ORDER SET st='PAID', sil=cobro.sillas, ttl=:ttl_largo
                                                      IF st IN (:pending, :expired)
        si pago ausente:    acciones += Put    PAY#nn (st='APPROVED', gtx, idem=oid#nn, amt)  IF
attribute_not_exists(pk)
        si pago != 'APPROVED': acciones += Update PAY#nn SET st='APPROVED', gtx REMOVE g3pk, g3sk
                                                      IF st IN (:initiated, :unknown)
        si inv ausente:     acciones += Put    INV#1 (st='PENDING', g3pk=PEND#INV#n)  IF
attribute_not_exists(pk)
        # NO se toca el contador de la localidad ni LIM#: se reconstruyen al final (§5.4)

        si acciones vacío: devolver YA_COMPLETA
        intentar:
            TransactWriteItems(acciones, ClientRequestToken = H(oid, "CONFIRM-D"))
            devolver CONFIRMADA
        capturar TransactionCanceledException:
            continuar                                          # releer y replanificar
    devolver ERROR_REVISAR_A_MANO                              # alerta; no se reembolsa a ciegas
```

5.3 Por qué esto es seguro

- **No hay doble venta.** Una silla pasa a `S` solo si está libre, vencida, ya es de esta orden o ya es vendida a esta orden. Si es de otra orden, el cobro va a reembolso. Aunque las ventas estén abiertas, ninguna compra nueva puede tomar una silla congelada, y solo escribe una región [D-07].
- **Es idempotente.** Se puede ejecutar dos veces (o por el webhook y por el reconciliador a la vez): el segundo plan solo encuentra lo que falta, y la condición `st = 'S' AND so = :oid` hace que repetir la silla no falle.
- **Cubre "PAID" a medias.** Miguel indica "si está `PAID`, no hacer nada". Con transacciones que no se replican como unidad [D-07], una orden en `PAID` puede tener la silla en `L` o la boleta sin crear. Por eso **también las órdenes `PAID` de la ventana pasan por `confirmarDesastre`**, que no hace nada si todo está completo (`YA_COMPLETA`) y repara si no.
- **Las boletas no se pisan.** Cada acción de boleta es `attribute_not_exists`: si la boleta ya estaba (incluso `USED`), no se toca.
- **Si no se pueden conocer las sillas** (sin metadatos): se reembolsa. Se pierde la venta, no el dinero ni la integridad.

5.4 Contadores: de "verificar" a "reconstruir"

Miguel propone verificar que la suma de fragmentos iguale las sillas `S`. Con transacciones a medias el contador puede haberse replicado sin la silla, o al revés, y `confirmarDesastre` no lo toca. Propongo: **al terminar la conciliación**, para cada localidad se cuentan las sillas `S` (consulta por filas, como el constructor del mapa) y se **fijan** los fragmentos (`SH#00` = el total, el resto en 0). Es válido porque el contador es **derivado** de las sillas (Jacobo, `entrega_para_el_equipo.md` §3.1). Como las ventas ya están abiertas, contar y fijar no es atómico: una venta que ocurra entre las dos operaciones puede dejar el contador corrido en unas pocas sillas. **Eso no afecta la doble venta**, porque la decisión de vender nunca usa el contador sino la silla, y el profesor acepta que lo que se muestra vaya con atraso (respuesta 5). El contador se reconstruye otra vez, exacto, cuando la preventa cierra. El límite por usuario `LIM#` se recalcula desde las órdenes, como indica Miguel.

5.5 Pagos que llegan después de abrir

Un cliente que ya estaba pagando cuando cayó la región puede ser aprobado **después** de reabrir. Para esos cobros, el consumidor de SQS usa `confirmarDesastre` con el mismo metadato **mientras la hora de creación de la orden sea anterior a la hora de declaración**. Para el resto, la confirmación normal.

6. Cuando vuelve la región original

Miguel pregunta si su lectura es correcta: la región original **no recibe tráfico ni escrituras** al volver y se concilia otra vez contra la pasarela. **La confirmo, con un matiz.**

- AWS dice que al volver "todas las replicaciones salientes y entrantes pendientes se reintentarán" y que "no se requiere ninguna acción especial" [D-08]. Eso es lo que necesitamos: **dejar que converja**, no intervenir a mano.
- El matiz es el criterio de "última escritura gana" [D-07]: las escrituras de la conciliación en us-east-2 son **posteriores** a las pendientes de us-east-1, así que normalmente ganan. Pero si us-east-1 tenía una versión más reciente de algún ítem que nunca llegó a replicarse, ese ítem podría volver con un estado distinto del que decidió la conciliación. Como estos conflictos no se registran [D-08], **se concilia otra vez contra la pasarela** después de la convergencia y antes de cualquier regreso a us-east-1, y el regreso se hace fuera de una preventa (Miguel).

7. Costo en rendimiento

Mecanismo	Efecto en la latencia de la compra	Costo adicional
Replicación en 3 zonas	Ninguno medible; ya está incluido en el "milisegundo de un dígito" del servicio [D-11]	Ninguno
MREC (elegido)	Ninguno en la escritura: la replicación es asíncrona [D-07]	Escrituras replicadas (USD 0,625 por millón en us-east-1, cifra de Jacobo, `entrega_para_el_equipo.md` §2.4)
MRSC (descartado)	Subiría la latencia de escritura (cruza regiones), según Jacobo (`arquitectura.md` §4.1)	No se puede usar con transacciones
Libro de pagos `bya_ledger` (MRSC, D-2b)	**Sí afecta:** una escritura sincrónica entre regiones antes de cada cobro y una actualización después. Latencia no medida (Jacobo, `modelo_datos.md` §3.9)	Tabla MRSC en tres regiones, 24.000 intentos/preventa; hasta 48.000 operaciones lógicas INTENT+CHARGED antes de reintentos (Juan Pablo)
Conciliación en modo desastre	No afecta la preventa normal	Horas de DBA; el enunciado asigna 6 horas por preventa (Miguel)
Pre-calentamiento de la réplica	—	Ninguno extra: se aplica solo a todas las réplicas (abajo)

Pre-calentamiento de la réplica: resuelto por Jacobo. Mi temor era que la réplica de us-east-2, que solo recibe tráfico de replicación, no soportara la carga de pico al conmutar. Jacobo encontró que en las tablas globales los valores de *warm throughput* "se aplicarán automáticamente a todas las réplicas" [R-18, D-18] (traducción mía). Basta con pre-calentar la tabla global al nivel del pico (≈ 10.546 WCU, cifra de Jacobo). Queda para Juan Pablo saber si el cargo del pre-calentamiento se cobra por región.

8. Pendientes

- **Pasarela:** elegida Mercado Pago (2026-11-03). Falta la prueba de sandbox de `pasarelas.md` §2 (Miguel, Jacobo, David).
- **Jacobo:** clave por minuto y modelo de costo del ledger ya añadidos en `modelo_datos.md` §3.9; falta medir latencia regional en AWS/desde Bogotá.
- **Miguel:** hecho el 2026-11-03 (runbook 2b de 10 pasos).
- **Profesor:** P-4 respondida el 2026-11-04: región en la nube, data center en Bogotá.
- **Miguel:** pasar el runbook 2b al congelamiento de sillas en duda (§5.0) y a la tabla de RTO de §3.3.
- **Simulacro:** ningún tiempo de este documento se ha medido; el plan de pruebas es de Miguel.

9. Referencias propias

Consultadas el 2026-11-03. Las citas en español son traducciones mías.

ID	Fuente	Qué respalda
D-07	How DynamoDB global tables work	Transacciones solo atómicas en la región de origen y visibles a medias en réplicas; escrituras condicionales contra la versión local; “última escritura gana”; lecturas fuertes posiblemente viejas; retraso típico ≤ 1 s y RPO de unos segundos; MRSC sin transacciones ni TTL y con tres regiones
D-08	Best practices for global table design	Al volver una región se reintentan las replicaciones pendientes sin acción especial; `ReplicationLatency` sin SLA; conflictos no registrados en CloudWatch ni CloudTrail; MREC con RPO y RTO “medidos en segundos”; sin cifra de conmutación
D-09	Routing strategies in DynamoDB	Modo *write to one Region*; TTL de DNS de 60 s típico y sus límites
D-10	Best practices for DynamoDB global tables, Part 2: failover strategies	RTO con ARC “segundos a pocos minutos” y con DNS “varios minutos”; alarma compuesta con `SystemErrors` y `ReplicationLatency`; ejemplo de 5xx > 5 % durante tres minutos; aumentar capacidad en la región destino; operaciones idempotentes y escrituras condicionales; pruebas con AWS Fault Injection Service
D-11	What is Amazon DynamoDB?	Replicación en 3 zonas y SLA 99,99 %; sin conmutaciones complicadas ni demoradas entre regiones; sin ventanas de mantenimiento
D-12	Amazon DynamoDB SLA	99,99 % estándar y 99,999 % para tablas globales; condiciones del segundo
D-13	How Route 53 determines the health of endpoints	Revisión cada 10 o 30 s; sin fórmula del tiempo total
D-14	Capital One: Moving to DynamoDB to increase application resiliency	Caso publicado: replicación ≈ 1 s en operación normal, mayor bajo carga; sin tiempo de conmutación absoluto
D-18	Understanding DynamoDB warm throughput	En tablas globales, el *warm throughput* se aplica automáticamente a todas las réplicas; la duración del pre-calentamiento depende de los valores y del tamaño de la tabla
D-22	Route 53 API: HealthCheckConfig	`FailureThreshold` entre 1 y 10, por defecto 3; `RequestInterval` de 10 o 30 s, por defecto 30